

école
normale
supérieure
paris-saclay

université
PARIS-SACLAY

RESEARCH PROJECT (TER) REPORT - SPRING 2026

Distributed framework for #SAT solvers

Thanh Phat Doan

Under the supervision of

Mihaela Sighireanu

Formal Methods Laboratory (LMF)

ENS Paris-Saclay

April 17, 2026

Abstract

Exact model counting (#SAT) is a fundamental problem in automated reasoning with applications in verification, diagnosis, and probabilistic inference. Although sequential model counters have become highly effective, many instances remain too expensive to solve on a single machine. This report presents a distributed framework for exact model counting based on a master-worker architecture. The framework partitions the search space into disjoint instances of the input problem (called “cubes”), assigns them to workers, and aggregates the resulting counts. To reduce load imbalance caused by heterogeneous subproblem difficulty, it combines adaptive timeout estimation with dynamic cube resplitting. The implementation is solver-agnostic and uses gRPC for communication between components. We evaluate the framework on standard benchmark instances and discuss its limitations and possible extensions.

Contents

1	Context	3
2	Motivation	3
3	Sequential #SAT solvers background	4
3.1	Alternative solvers	4
3.2	Ganak2	4
4	My contribution	5
4.1	Work partitioning via cube decomposition and dynamic load balancing	5
4.2	Master-side	7
4.3	Worker-side	9
5	Implementation and evaluation	10
5.1	Implementation	10
5.2	Evaluation	12
5.2.1	Experimental Setup	12
5.2.2	Results	12
5.2.3	Discussion	12
6	Conclusion and future work	14
6.1	Conclusion	14
6.2	Limitations and improvements	14
	Bibliography	15

1. Context

The Boolean Satisfiability Problem (**SAT**) represents the foundational challenge in automated reasoning, seeking to determine if there exists an assignment of truth values to variables that renders a given propositional formula true. These formulas are typically expressed in Conjunctive Normal Form (**CNF**), which consists of a conjunction of clauses, each representing a disjunction of literals. Consider the formula in CNF form $F = (x1 \vee x2) \wedge (\neg x1 \vee x3) \wedge (\neg x2 \vee \neg x3)$. A SAT solver aims to identify a satisfying assignment, such as $\{x1 = \top, x2 = \perp, x3 = \top\}$, which makes the formula evaluate to true.

This assessment of satisfiability is crucial across various domains, including hardware and software verification[1]. For instance, verifying that a software program adheres to its specifications can be translated into a SAT problem, where the program’s behavior is encoded as a propositional formula. Because of such wide-ranging applications, despite the NP-completeness of SAT (**Cook’s theorem** [2]), there has been significant progress in developing efficient solvers (notably CaDiCaL solver [3]) that can handle large and complex instances using sophisticated algorithmic techniques like Conflict-Driven Clause Learning (CDCL), chronological and non-chronological backtracking [4] to efficiently navigate and reduce the search space.

While SAT addresses the existence of a solution, Model Counting (**#SAT**) extends this problem by seeking to determine the total number of satisfying assignments within the 2^n possible configurations of a formula with n variables. This shift from decision to quantification elevates the problem to #P-complete [5], introducing a significantly higher level of computational complexity. However, #SAT enables a deeper analysis than classical SAT, providing insights into the solution space’s structure and size. Particularly in the case of software verification [6], the number of satisfying assignments can be used to estimate the proportion of inputs for which the program behaves correctly, providing a probabilistic measure of correctness. Another prominent application of #SAT is in security analysis, where it can be used to evaluate the robustness of cryptographic protocols [7] by not only determining if a vulnerability exists but also counting the number of ways an adversary can compromise a system and therefore assessing the risk level.

2. Motivation

Despite the advancements in current state-of-the-art #SAT solvers, many real-world instances remain computationally impractical for single-machine solving due to the exponential growth of the search space. Therefore, there is a pressing need for exploration of parallelization techniques in order to satisfy the demand of large-scale industrial benchmarks (e.g., circuit verification with millions of variables) or time-sensitive applications (e.g., probabilistic inference in machine learning). Unfortunately, as more complex algorithms and heuristics are developed for model counting, it is not trivial to adapt them to a distributed setting since they often rely on intricate interactions between components (component caching, clause learning, etc).

However, recent work has shown that model counting does exhibit parallel characteristics when the search space is appropriately decomposed. The cube-and-conquer paradigm [8] involves partitioning the search space into smaller “cubes” (partial assignments) and solving each cube independently, which can be effectively parallelized across multiple workers as demonstrated in [9]. This approach has shown significant speedups compared to sequential solving. However, [9] has only explored static strategies for cube generation and distribution, especially with adaptive strategies for handling heavy-tailed distributions of subproblem difficulty, where some subproblems may be solved very quickly, while others may require

significantly more time.

Accurately estimating the computational difficulty of these subproblems before solving them is challenging, and static partitioning strategies may lead to load imbalance and resource underutilization. Therefore, to address these problems, we propose the implementation of adaptive timeout mechanisms and dynamic work redistribution strategies. Furthermore, we intend to move beyond the constraints of traditional Message Passing Interface (**MPI**) by developing a flexible framework that supports heterogeneous deployment across mixed environments. By leveraging **gRPC** for inter-node communication, the solver can seamlessly integrate diverse resources, ranging from High-Performance Computing (HPC) clusters and cloud-based virtual machines to local workstations, into a unified distributed solving process. This modular architecture treats the underlying model counters as black boxes, allowing the framework to be easily adapted to any solver by simply wrapping its input and output interfaces within the distributed messaging layer.

3. Sequential #SAT solvers background

The efficiency of a distributed framework for #SAT solving is tied to the performance of its sequential backends. This chapter provides an overview of the current landscape of model counters and details the architecture of Ganak2, which serves as the primary engine for our framework.

3.1 Alternative solvers

The field of model counting has branched into several distinct algorithmic families. While our framework is designed to be solver-agnostic, that is it can accept any engine that follows the standard DIMACS CNF [10] format, understanding these alternatives is crucial for selecting the right backend for specific problem domains.

- **d4** : a knowledge compilation-based counter that compiles formulas into decision-DNNFs [11]
- **SharpSAT-TD** [12] : an evolution of the classic SharpSAT [13] that leverages **tree-decomposition** [14]. By identifying the structural width of the formula’s graph, it optimizes the order of component processing, often outperforming standard counters on industrial benchmarks with high regularity.
- **ApproxMC** [15] : an approximate model counter that provides probabilistic guarantees on the count. It is particularly useful for large instances where exact counting is infeasible or when an estimate is sufficient for the application at hand, offering a trade-off between accuracy and computational resources.

3.2 Ganak2

For the primary evaluation of this work, Ganak2 [16, 17] is chosen and utilized as the core sequential backend. Ganak2 is a state-of-the-art model counter that builds upon the sharpSAT architecture. Here’s a high-level overview of its architecture and key features:

Dual Independent Set Framework Utilizing the integrated **Arjun** [18] preprocessor, Ganak2 implements a dual-set strategy to balance search efficiency with SAT-solver transitions. It distinguishes between the **S-set** (SAT-eligibility set) and the **D-set** (Decision set):

- **S-set** ($S \subseteq P$): This is the minimal independent support identified by Arjun. Any two assignments agreeing on S define the values of all variables in the projection set P . Ganak2 uses this set as a threshold: once all variables in S are assigned, the solver can terminate the counting search and switch to a standard SAT solver to check the remaining formula.

- **D-set** ($D \supseteq P$): Unlike traditional counters that branch only on P , Ganak2 allows a larger decision set. By branching on variables outside the minimal support, the solver can more effectively decompose the formula into independent components, significantly increasing the “hit rate” of the component cache.

Formula Simplification Arjun employs two distinct phases to identify dependent variables: syntactic gate recovery (detecting AND and XOR gates) and implicit definability checks based on **Padoa’s Theorem** [19]. By leveraging an assumption-based CDCL framework, it prunes the formula of redundant variables before the search begins, preventing the solver from performing unnecessary computations on variables that are already logically defined by the support set.

Refined Residual Formula Processing Ganak2 optimizes the leaves of the search tree by switching to a modern SAT solver engine when the residual formula becomes simple enough. Crucially, Ganak2 maintains a seamless state transition, allowing the solver to move between the counting phase and the SAT phase without losing learned clauses or forcing an expensive restart of the component cache.

The latest 2026 iteration of Ganak2 introduces native intra-node parallelism. Our framework invokes Ganak2 via the command line, typically using the following structure:

```
ganak2 formula.cnf --threads N
```

This creates a two-tier parallelization strategy: our framework manages inter-node distribution (across the cluster), while Ganak2 manages intra-node efficiency through multi-threading.

4. My contribution

This section presents the design and implementation of a distributed framework for exact model counting. The system is built around three principles. First, the search space is decomposed into independent cubes so that subproblems can be solved in parallel. Second, task execution is controlled through an adaptive timeout mechanism that reacts to the observed difficulty of the instance. Third, the framework remains solver-agnostic by treating the underlying model counter as an external backend accessed through a uniform interface.

The architecture follows a master-worker model. The master process is responsible for generating cubes, distributing tasks, collecting results, updating the global model count, and refining the timeout policy. Workers receive cubes, invoke a sequential solver on the corresponding restricted formulas, and report back either a model count or a timeout event. When a task exceeds the current deadline, the cube is split into smaller subcubes and returned to the work queue. This design allows the framework to adapt to instances with highly uneven subproblem difficulty while preserving the modularity needed to support different solvers and deployment environments.

The remainder of this chapter formalizes the cube decomposition strategy, describes the master and worker protocols, and explains the heuristics used for task redistribution.

4.1 Work partitioning via cube decomposition and dynamic load balancing

Cube decomposition Let F be a CNF formula over a set of variables X . A cube C is a conjunction of literals over a subset of X , that is,

$$C = \ell_1 \wedge \ell_2 \wedge \cdots \wedge \ell_k,$$

where each ℓ_i is a literal and no variable appears twice with conflicting polarity. The restricted counting problem associated with C is the formula $F \wedge C$.

A set of cubes $\mathcal{C} = \{C_1, \dots, C_m\}$ forms a partition of the search space if the cubes are pairwise contradictory and collectively exhaustive. In other words, for all $i \neq j$,

$$C_i \wedge C_j \equiv \perp,$$

and

$$\bigvee_{i=1}^m C_i \equiv \top$$

over the assignments that remain unspecified at the current level of decomposition. Under these conditions, exact model counting is additive:

$$\#(F) = \sum_{i=1}^m \#(F \wedge C_i).$$

This property makes cube decomposition particularly suitable for distributed solving, since each worker can process one subproblem independently and the final result is obtained by summing the partial counts.

In practice, however, the cost of solving the subproblems is not uniform. Some cubes are resolved quickly, while others lead to expensive search trees. This heavy-tailed behavior can create significant load imbalance if the decomposition is purely static. To address this issue, the framework uses adaptive timeout control together with dynamic resplitting of cubes that exceed their allotted time.

Adaptive timeout Let S denote the set of cubes that have already been solved successfully, and let $t(C)$ denote the solving time of cube C . The timeout threshold is updated from the observed runtime history:

$$T_{\text{limit}} = \alpha \cdot \frac{1}{|S|} \sum_{C_i \in S} t(C_i),$$

where $\alpha > 0$ is a scaling factor. This policy allows the timeout to evolve with the actual difficulty of the benchmark rather than remaining fixed throughout the execution. At the beginning of a run, the timeout may be initialized to $+\infty$ until enough observations are available to compute a meaningful estimate.

Branching heuristics We define a heuristic function

$$h : (F, C) \mapsto (x_1, \dots, x_k), \quad 1 \leq k \leq k_{\max},$$

which selects a small set of branching variables from the restricted formula $F \wedge C$. The parameter $k_{\max}$ is kept small (typically $k \in \{1, 2, 3\}$) to avoid excessive splitting which can lead to an explosion of new cubes in the queue.

The heuristic can be instantiated in several ways depending on the desired trade-off between computational cost and decomposition quality. For example, it may select variables uniformly at random, prioritize variables with high clause occurrence, exploit structural properties in tree decompositions [14], or reuse solver-specific signals such as VSIDS scores. The goal is to identify variables whose assignment is likely to significantly simplify the remaining formula.

Cube splitting function Given $(x_1, \dots, x_k) = h(F, C)$, the cube C is refined by enumerating all assignments to the selected variables. For each vector $\sigma \in \{0, 1\}^k$, define:

$$\ell(\sigma) = \bigwedge_{i=1}^k \begin{cases} x_i & \text{if } \sigma_i = 1, \\ \neg x_i & \text{if } \sigma_i = 0. \end{cases}$$

The splitting function is then defined as:

$$f_{\text{split}}(F, C) = \{C \wedge \ell(\sigma) \mid \sigma \in \{0, 1\}^k\}.$$

This construction ensures that the resulting cubes form a disjoint partition of the original cube:

$$\bigvee_{C' \in f_{\text{split}}(F, C)} C' \equiv C, \quad C' \wedge C'' \equiv \perp \text{ for } C' \neq C''.$$

This formulation generalizes classical binary branching. When $k = 1$, the cube is split into two subcubes, whereas larger values of k produce 2^k subcubes in a single step. In contrast to static decomposition strategies, splitting is applied selectively only to cubes that are empirically hard, as indicated by timeout events. This avoids unnecessary overhead on easy subproblems while allowing finer-grained parallelism where it is most needed.

However, this redistribution approach faces a significant inherent challenge: the loss of accumulated work of restarting a solver on a new cube. When a worker abandons a cube due to a timeout, the progress made during that search is lost unless the solver’s internal state, such as cached components or learned clauses. While such recovery is technically feasible, it often requires dissecting the internal particularities of specific solvers, which is outside the scope of our solver-agnostic framework. To mitigate this lost work penalty, we aim to minimize timeout occurrences by carefully calibrating the granularity of the initial decomposition. By generating cubes that are sufficiently large to justify the overhead of a distributed start, yet fine-grained enough to ensure that the relative difficulty between subproblems remains balanced, we can reduce the frequency of re-splitting and maximize overall throughput.

4.2 Master-side

The framework follows a master-worker architecture, where a central coordinator (master) is responsible for generating the cubes (tasks) and distributing them to a pool of worker nodes. The master operates in a fully asynchronous, event-driven manner, where all interactions are triggered by **RPC** calls from workers. Task distribution is handled through the **GetTask** procedure: when a worker becomes idle, it requests a new cube from the master. If no tasks are available, it receives a special payload having *task_id* set to -1 , signaling it to wait for a bit and retry later.

Upon receiving a result via **SubmitResult**, the master distinguishes between completed and timed-out tasks. Successful executions contribute directly to the global model count and are recorded to update the dynamic timeout. The timeout T_{limit} is recomputed as a scaled average of previously observed solving times, allowing the system to adapt to the hardness of the instance over time. Updated timeout values are propagated to workers through a streaming RPC mechanism. It should be noted that the master doesn’t have to broadcast constantly the new updated timeout value if the change is not significant enough (e.g., less than 10% difference from the previous value) to avoid unnecessary communication overhead and network congestion.

In the case of a timeout, the corresponding cube is refined using the splitting function f_{split} guided by specific heuristics to generate a set of smaller subcubes. These are then reinserted into the queue, from which they will be distributed to workers as they become available.

The architecture naturally supports scalability as the worker pool is not fixed and can be extended dynamically as new workers establish gRPC connections to the master. Since workers independently pull tasks and report results, the system achieves load balancing without prior knowledge of available computational resources.

Algorithm 1: Master

Input: F : CNF formula;
 f_{split} : cube splitting function;
 T_{limit} : initial timeout;
 α : timeout scaling factor
Output: $\text{totalCount} = \#(F)$

Parse F ; // read the input instance and initialize the root problem
Initialize queue $Q \leftarrow$ initial cubes ; // seed the work queue
Initialize solved set $S \leftarrow \emptyset$; // store completed cubes for timeout updates
 $\text{totalCount} \leftarrow 0$; // global exact model count
Start gRPC server (SolverService) ; // enable asynchronous worker communication
while *true* **do**
 // RPC: GetTask from worker
 if *Receive GetTask*(w) **then**
 if $Q = \emptyset$ **then**
 send TaskPayload($\text{task_id} \leftarrow -1$) to w ; // no work currently available
 else
 $c \leftarrow$ dequeue(Q) ; // assign the next pending cube
 generate unique task_id ; // track the task across retries
 send TaskPayload($\text{task_id}, c, F, T_{\text{limit}}$) to w ; // dispatch task and deadline
 // RPC: SubmitResult from worker
 if *Receive SubmitResult*($\text{task_id}, w, \text{count}, t, \text{timed_out}$) **then**
 if $\text{timed_out} = \text{false}$ **then**
 $\text{totalCount} \leftarrow \text{totalCount} + \text{count}$; // accumulate exact count
 $S \leftarrow S \cup \{t\}$; // record observed runtime
 $T_{\text{limit}} \leftarrow \alpha \cdot \frac{1}{|S|} \sum_{t_i \in S} t_i$; // update adaptive timeout
 broadcast TimeoutUpdate(T_{limit}) ; // notify workers of the new deadline
 else
 $\{C_1, \dots, C_k\} \leftarrow f_{\text{split}}(F, c)$; // refine the timed-out cube
 foreach C_j **do**
 enqueue(Q, C_j) ; // return refined tasks to the queue
return totalCount

4.3 Worker-side

The worker follows a pull-based execution model, where tasks are requested from the master on demand via the `GetTask` RPC. This eliminates the need for centralized scheduling and allows workers to operate independently. Each task is executed by spawning an external solver process, which is invoked with the appropriate command-line arguments to solve the restricted formula $F' \equiv F \wedge C$. This process-level isolation enables safe termination and restart of the solver in response to timeout conditions.

To support adaptive execution, workers subscribe to a streaming RPC interface `SubscribeTimeoutUpdates` and continuously receive updated timeout values from the master. These updates are monitored asynchronously, and if a new timeout is exceeded during execution, the worker terminates the solver process early and reports the task as timed out.

The worker also emits periodic status updates containing runtime metrics such as CPU and memory usage. These updates are used solely for monitoring and visualization purposes and do not influence the solving process.

Finally, workers can join or leave the system at any time by establishing or closing a gRPC connection to the master. Once the master detects a new worker connection, it adds it to the pool of available workers and starts assigning tasks to it. If a formula has been solved, the master can still take in a new problem and start the process again with current workers without any disruption. Since each worker is fully decoupled from the others and manages its execution and communicates to the master only through RPC interfaces, the system remains robust and adaptable to varying computational environments.

Algorithm 2: Worker

```

Input:  $A$ : master address;
 $w$ : worker identifier
Output: None
Connect to master at  $A$  ;                                // establish gRPC session
while true do
     $task \leftarrow \text{GetTask}(w)$  ;                            // request a new cube
    if  $task.task\_id = -1$  then
        sleep for a fixed interval ;                        // no work is currently available
        continue
    Construct restricted formula  $F \wedge C$  from  $task$  ; // instantiate the assigned subproblem
    Launch solver process  $P$  ;                               // run the sequential backend in isolation
    Initialize timer  $t \leftarrow 0$  ;                         // track elapsed time
    Initialize  $timed\_out \leftarrow \text{false}$  ;                // task status flag
    start listener for TimeoutUpdate stream ; // receive deadline updates asynchronously
    while  $P$  is running do
        if  $t \geq task.timeout$  then
            terminate  $P$  ;                                  // enforce the local deadline
             $timed\_out \leftarrow \text{true}$  break
        if new timeout  $T_{limit}$  received then
            if  $t \geq T_{limit}$  then
                terminate  $P$  ; // stop early if the updated deadline is exceeded
                 $timed\_out \leftarrow \text{true}$  break
            send ReportStatus( $w, task, t, \text{resource usage}$ ) ; // publish monitoring data
            update elapsed time  $t$ 
        if  $timed\_out = \text{false}$  then
            count  $\leftarrow$  parse solver output ;           // recover the exact local count
        else
            count  $\leftarrow 0$  ;                               // timed-out tasks contribute no result
    SubmitResult( $task.id, w, \text{count}, t, timed\_out$ ) ;      // report completion

```

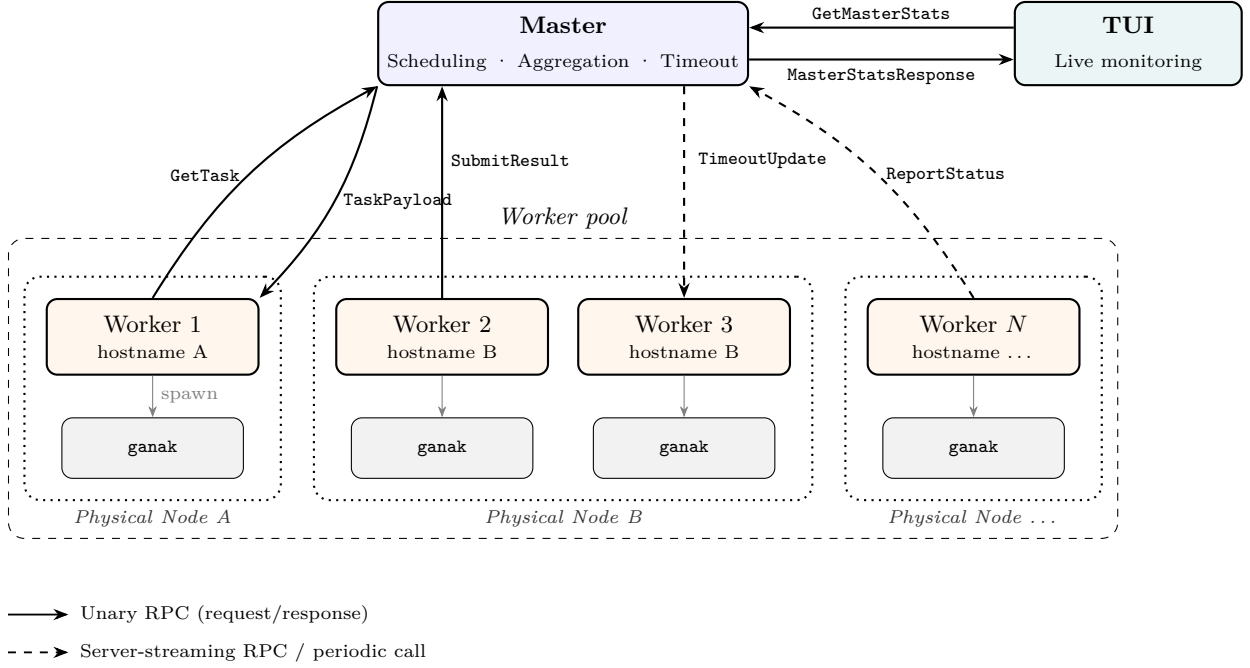


Figure 5.1: Distributed master-worker architecture.

5. Implementation and evaluation

5.1 Implementation

General Architecture Our framework is built upon a distributed master-worker architecture designed to handle large-scale parallelized model counting. The system leverages gRPC for high-performance communication, utilizing a pull-based task distribution model where workers request formulas and the master provides centralized scheduling and result aggregation. As illustrated in Figure 5.1, this design incorporates real-time telemetry via bidirectional streams and a dedicated TUI for live monitoring of the global solving state.

Communication layer The distributed components communicate through gRPC and Protocol Buffers. This choice provides a strongly typed interface between the master, the workers, and the terminal user interface. The protocol definitions are compiled into native client and server code, which reduces serialization errors and avoids the overhead of text-based formats. gRPC also supports bidirectional streaming, which is useful for continuous status updates during solving without blocking the main request-response flow. Since the system is intended to run across heterogeneous environments, the use of standard TCP/IP communication makes it possible to deploy workers on different machines, operating systems, and containerized environments without requiring a homogeneous cluster.

Master implementation The master is implemented in Go. Go is well suited to the coordination layer because its concurrency model makes it straightforward to manage many simultaneous worker connections. Goroutines and channels provide lightweight concurrency primitives, while the standard library offers direct support for gRPC, synchronization, cancellation, and arbitrary-precision arithmetic. In addition, Go produces a single statically linked binary, which simplifies deployment and makes the master process

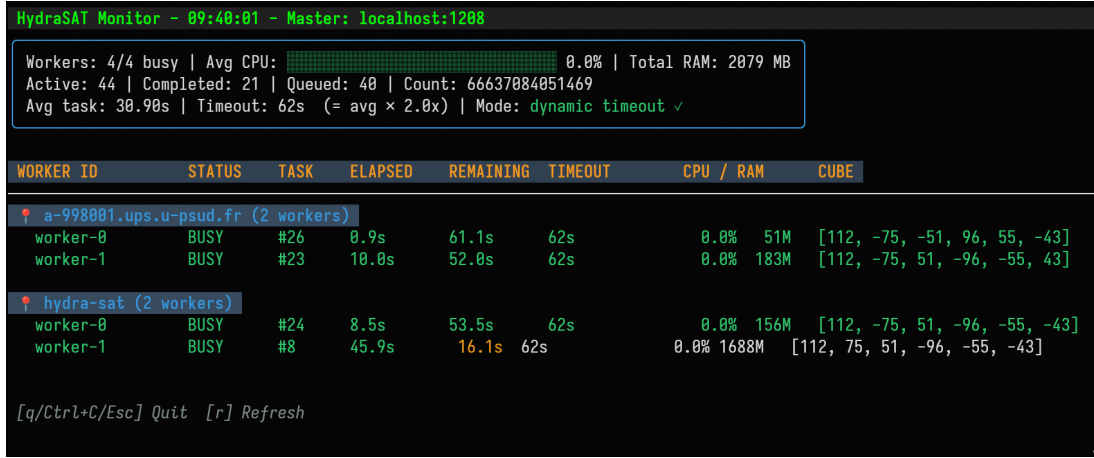


Figure 5.2: Terminal user interface showing real-time statistics of the distributed solving process. The top panel displays global metrics such as current total model count, active workers, and timeout thresholds. Workers are separated by groups of physical nodes, with individual status indicators for each worker and its current task.

suitable for long-running server execution.

Worker implementation The workers are implemented in Python. This choice is motivated by the ease of process control, since the solver is executed as an external subprocess whose standard output, errors, and lifetime must be monitored and controlled. Python also provides convenient access to system-monitoring libraries and simplifies experimentation with different solver integrations. Because the worker logic mainly orchestrates execution rather than performing heavy computation itself, Python offers a practical balance between flexibility and implementation speed.

Containerization The system is packaged with Docker to ensure reproducibility and simplify deployment. Each worker can run inside an isolated container with the solver pre-installed, which avoids dependency conflicts and helps guarantee consistent execution environments. Docker Compose is used to orchestrate the master, the worker replicas, and the supporting services. This approach supports both fully containerized deployments and hybrid setups in which only part of the system is containerized.

Terminal user interface The terminal user interface (cf. Figure 5.2) is implemented in Go using Bubble Tea [20]. Its architecture follows the event-driven style of the Elm framework: the model stores the current state of the system, updates react to events such as user input or gRPC responses, and the view renders the state to the terminal. This structure is appropriate for a monitoring interface because it keeps rendering separate from data collection and prevents the interface from blocking while waiting for network responses.

Build system A Makefile coordinates protocol generation, compilation, and deployment. It automates code generation from the Protocol Buffer definitions, builds the Go binaries, and launches the distributed environment with a single command. This reduces manual setup and ensures that the master, the workers, and the interface remain synchronized with the same communication schema.

5.2 Evaluation

5.2.1 Experimental Setup

Due to hardware limitations we could not yet conduct a comprehensive evaluation on large-scale industrial benchmarks. Instead, we performed a preliminary study on randomly generated 3-SAT instances to validate the distributed counting pipeline and characterise its scaling behaviour.

Hardware. All experiments used two virtual machines, each allocated 8 CPU cores, constituting a **16-core worker pool** in the largest configuration. Workers were deployed as independent Python processes, each driving one *Ganak2* subprocess as its backend solver.

Benchmark instances. The instances are 3-SAT formulas generated by **cnfgen** [21]. Since it is difficult to predict how long a given instance will take to solve, we filtered out instances that exceed the one-hour time limit when solved sequentially with *Ganak2*. We selected 4 instances with varying difficulty levels for this evaluation, denoted as I_1 to I_4 in order of increasing runtime. Trivial UNSAT generated instances benchmark is also discussed in Section 5.2.3.

Distributed configuration. The initial cube decomposition was obtained by branching on the 6 most frequent variables in the original formula (producing $2^6 = 64$ initial cubes). Sub-cubes that exceeded the dynamic timeout were split further using a simple frequency-based selector: the variable appearing most often in the formula that had not yet been assigned in the cube was chosen as the branching point. The dynamic timeout scaling factor was set to $\alpha = 2$, so a new timeout of $\lfloor 2\bar{t} \rfloor$ seconds was issued to all workers whenever the rolling average $\bar{t}$ of completed-task durations changed.

Configurations compared. Three configurations are evaluated:

- **Sequential** — a single *GANAK 2* process solving the full formula directly (1 core, no framework overhead)
- **#SAT-8** — the distributed framework with 8 workers
- **#SAT-16** — the distributed framework with 16 workers

5.2.2 Results

Wall-clock times and the resulting speedups over the sequential baseline are reported in Table 5.1. Figure 5.3 plots the same data on a logarithmic axis to make both the fast and slow instances legible on the same scale.

5.2.3 Discussion

Overall speedup. The distributed framework consistently outperforms sequential solving on all four SAT instances. Across the two distributed configurations, speedups range from $1.7\times$ to $11.1\times$, confirming that the divide-and-count approach is effective on this class of benchmarks.

Table 5.1: Wall-clock times (seconds) and speedup relative to sequential GANAK 2 for four 3-SAT instances. Speedup > 1 indicates improvement over the baseline.

Instance	Sequential	#SAT-8		#SAT-16	
	Time (s)	Time (s)	Speedup	Time (s)	Speedup
I_1	723.4	73.8	$9.8\times$	65.0	$11.1\times$
I_2	2350.7	870.8	$2.7\times$	540.3	$4.4\times$
I_3	505.2	206.2	$2.4\times$	208.4	$2.4\times$
I_4	1967.4	1151.7	$1.7\times$	756.5	$2.6\times$

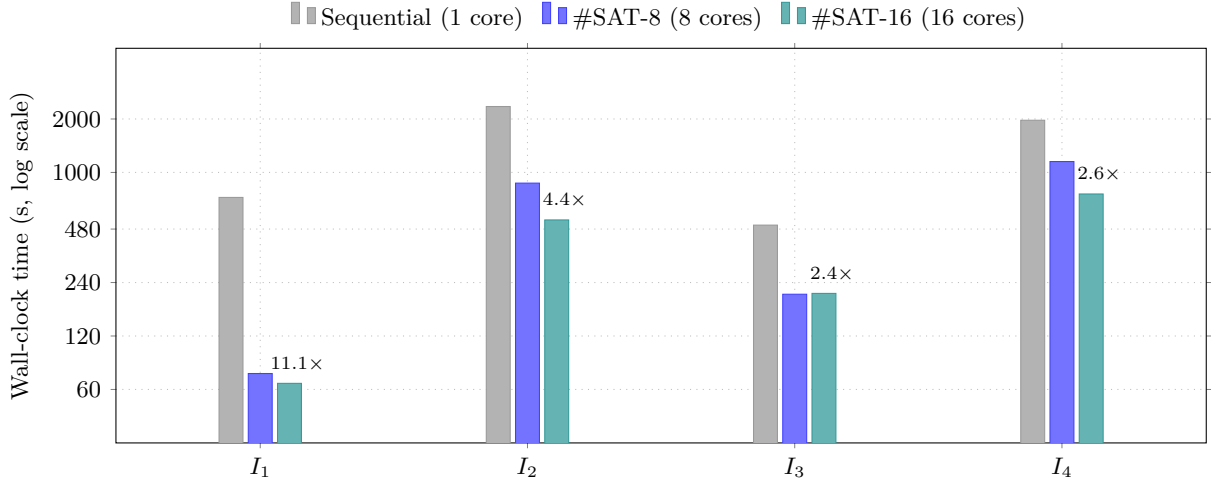


Figure 5.3: Wall-clock time comparison on a logarithmic scale. Speedup labels above the #SAT-16 bars are relative to the sequential baseline. Bars below the sequential bar indicate a speedup; the smaller the bar, the faster.

Non-linear and instance-dependent scaling. The speedup is far from uniform. Instance I_1 achieves a near-ideal $11.1\times$ speedup with 16 cores, suggesting that the 64 initial cubes were well-balanced and that most sub-problems required comparable solving time. Instances I_2 and I_4 , by contrast, yield only $4.4\times$ and $2.6\times$ respectively at 16 cores despite the same configuration. This disparity points to an imbalanced work distribution in which a small number of cubes dominate total runtime, and additional workers cannot accelerate them because they are still waiting for a single bottleneck sub-task to complete or time out. Using much more advanced splitting heuristics that take into account the structure of the formula and the behaviour of the solver on previous cubes is a promising direction to mitigate this issue.

Marginal gain from 8 to 16 cores. Instance I_3 is a notable outlier: both the 8- and 16-core configurations finish in almost identical times (206s vs. 208s). A likely explanation is that the bottleneck lies not in worker count but in the timeout parameter: if one or two difficult sub-cubes dominate, increasing parallelism only changes which workers are idle, not when the last cube finishes. Fine-tuning α per problem class or implementing a more sophisticated timeout policy that considers the distribution of observed runtimes could help mitigate this problem and allow better scaling with more workers.

Highly-constrained and unsatisfiable instances. We have also experimented with formulas near or above the phase-transition threshold, which are often *unsatisfiable* and are trivially recognised as such by Ganak2 through unit propagation and conflict analysis in under one or two seconds. In this regime the distributed framework is *slower* than sequential: the gRPC round-trip, cube-queue management, and the initial 64-cube expansion add 6–12 seconds of overhead that dwarf the actual solving time. For benchmarks where a fast SAT or UNSAT decision is expected, running the sequential solver directly

and falling back to the distributed framework only on timeout is a straightforward strategy that would eliminate this overhead entirely.

Summary. Our framework demonstrates clear speedups on satisfiable instances of moderate hardness, with the best result reaching $11.1\times$ at 16 cores. The principal bottlenecks identified are unbalanced cube decomposition and suboptimal timeout calibration. Both are easy targets for future refinement, and we expect that significant further speedups are possible with more sophisticated heuristics and adaptive policies.

6. Conclusion and future work

6.1 Conclusion

Our framework addresses the scalability limitations of sequential model counting through a modular, gRPC-based distributed framework designed for heterogeneous environments. By decoupling the coordination layer from the underlying sequential engines, the framework maintains a solver-agnostic architecture that can integrate any DIMACS-compatible tool, such as Ganak2 or d4. The primary contribution of this work lies in the implementation of adaptive mechanisms, specifically dynamic cube resplitting and a self-calibrating timeout strategy, to mitigate the performance bottlenecks caused by heavy-tailed distributions of subproblem difficulty. Preliminary results indicate that while the framework achieves significant speedups.

6.2 Limitations and improvements

Preliminary results indicate that the framework can reduce overall solving time on instances with uneven subproblem difficulty, although its effectiveness highly depends on the heuristic used.

A primary area of improvement lies in the cube decomposition process, where the integration of structural heuristics such as tree decomposition or VSIDS-based [22](Variable State Independent Decaying Sum) variable selection could facilitate the generation of more balanced and higher-quality subproblems.

Furthermore, while the current dynamic timeout mechanism relies on a simple average of historical execution times, a more robust probabilistic model could be developed to estimate T_{limit} based on the structural similarities between active and previously solved cubes.

To further optimize resource allocation, a "scouting" mechanism could be implemented where a subset of workers utilizes fast SAT heuristics to gauge the complexity of subproblems that appear stagnant. This approach would allow the system to intelligently distinguish between truly intractable sub-cubes and those nearing completion, enabling more informed decisions on whether to trigger a redistribution or grant a "mercy" extension to the current worker.

Beyond heuristic refinements, long-term research will explore the integration of distributed hash tables or databases to facilitate component cache sharing across the network. This would require a more deep integration with the internal workings of the underlying solvers, but it has the potential to significantly reduce redundant computations by allowing workers to benefit from the progress of their peers.

Additionally, a portfolio-based approach could be implemented to deploy multiple solver types simultaneously, allowing the framework to automatically select the most efficient backend for a given sub-cube.

Bibliography

- [1] Franjo Ivančić et al. “Efficient SAT-based bounded model checking for software verification”. In: *Theoretical Computer Science* 404.3 (2008). International Symposium on Leveraging Applications of Formal Methods (ISoLA 2004), pp. 256–274. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2008.03.013>. URL: <https://www.sciencedirect.com/science/article/pii/S0304397508002223>.
- [2] Stephen A. Cook. “The complexity of theorem-proving procedures”. In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. STOC ’71. Shaker Heights, Ohio, USA: Association for Computing Machinery, 1971, pp. 151–158. ISBN: 9781450374644. DOI: [10.1145/800157.805047](https://doi.org/10.1145/800157.805047). URL: <https://doi.org/10.1145/800157.805047>.
- [3] Armin Biere et al. “CaDiCaL 2.0”. In: *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I*. Ed. by Arie Gurfinkel and Vijay Ganesh. Vol. 14681. Lecture Notes in Computer Science. Springer, 2024, pp. 133–152. DOI: [10.1007/978-3-031-65627-9_7](https://doi.org/10.1007/978-3-031-65627-9_7).
- [4] Robin Coutelier. “Chronological vs. non-chronological backtracking in satisfiability modulo theories”. Unpublished master’s thesis. MA thesis. Liège, Belgique: Université de Liège, 2023. URL: <https://matheo.uliege.be/handle/2268.2/18041>.
- [5] Hans van Maaren Armin Biere Marijn Heule and Toby Walsch. “Handbook of Satisfiability”. In: (2008). URL: <https://www.cs.cornell.edu/~sabhar/chapters/ModelCounting-SAT-Handbook-prelim.pdf>.
- [6] Samuel Teuber and Alexander Weigl. “Quantifying Software Reliability via Model-Counting”. In: *Quantitative Evaluation of Systems*. Ed. by Alessandro Abate and Andrea Marin. Cham: Springer International Publishing, 2021, pp. 59–79. ISBN: 978-3-030-85172-9.
- [7] Guilhem Lacombe and Sébastien Bardin. “Attacker Control and Bug Prioritization”. In: *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*. Ed. by Lujo Bauer and Giancarlo Pellegrino. USENIX Association, 2025, pp. 4541–4560. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/lacombe>.
- [8] Marijn Heule et al. “Cube and conquer: guiding CDCL SAT solvers by lookaheads”. In: vol. 7261. Dec. 2011, pp. 50–65. ISBN: 978-3-642-34187-8. DOI: [10.1007/978-3-642-34188-5_8](https://doi.org/10.1007/978-3-642-34188-5_8).
- [9] Zhenghang Xu, Minghao Yin, and Jean Marie Lagniez. “An Embarrassingly Parallel Model Counter”. In: *Proceedings of the 22nd International Conference on Principles of Knowledge Representation and Reasoning*. Oct. 2025, pp. 670–681. DOI: [10.24963/kr.2025/65](https://doi.org/10.24963/kr.2025/65). URL: <https://doi.org/10.24963/kr.2025/65>.
- [10] *SAT Competition 2009: Benchmark Submission Guidelines*. <https://web.archive.org/web/20190325181937/https://www.satcompetition.org/2009/format-benchmarks2009.html>. Accessed: 202X-XX-XX. 2009.
- [11] Umut Oztok and Adnan Darwiche. “On Compiling CNF into Decision-DNNF”. In: *Principles and Practice of Constraint Programming*. Ed. by Barry O’Sullivan. Cham: Springer International Publishing, 2014, pp. 42–57. ISBN: 978-3-319-10428-7.
- [12] Tuukka Korhonen and Matti Järvisalo. *SharpSAT-TD in Model Counting Competitions 2021-2023*. 2023. arXiv: [2308.15819](https://arxiv.org/abs/2308.15819) [cs.AI]. URL: <https://arxiv.org/abs/2308.15819>.

- [13] Marc Thurley. “sharpSAT: counting models with advanced component caching and implicit BCP”. In: *Proceedings of the 9th International Conference on Theory and Applications of Satisfiability Testing*. SAT’06. Seattle, WA: Springer-Verlag, 2006, pp. 424–429. ISBN: 3540372067. DOI: [10.1007/11814948_38](https://doi.org/10.1007/11814948_38). URL: https://doi.org/10.1007/11814948_38.
- [14] Tuukka Korhonen and Matti Järvisalo. “Integrating Tree Decompositions into Decision Heuristics of Propositional Model Counters”. In: *27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*. Ed. by Laurent D. Michel. Vol. 210. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 8:1–8:11. ISBN: 978-3-95977-211-2. DOI: [10.4230/LIPIcs.CP.2021.8](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CP.2021.8). URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CP.2021.8>.
- [15] Yash Pote, Kuldeep S. Meel, and Jiong Yang. “Towards Real-Time Approximate Counting”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 39.11 (Apr. 2025), pp. 11318–11326. DOI: [10.1609/aaai.v39i11.33231](https://ojs.aaai.org/index.php/AAAI/article/view/33231). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/33231>.
- [16] Mate Soos and Kuldeep S. Meel. “Engineering an Efficient Probabilistic Exact Model Counter”. In: *Proceedings of the International Conference on Computer Aided Verification (CAV)*. 2025.
- [17] Shubham Sharma et al. “GANAK: A Scalable Probabilistic Exact Model Counter”. In: *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*. 2019.
- [18] Mate Soos and Kuldeep S. Meel. “Arjun: An Efficient Independent Support Computation Technique and its Applications to Counting and Sampling”. In: *CoRR* abs/2110.09026 (2021). arXiv: [2110.09026](https://arxiv.org/abs/2110.09026). URL: <https://arxiv.org/abs/2110.09026>.
- [19] Alonzo Church. “Alessandro Padoa. Essai d’une théorie algébrique des nombres entiers, précédé d’une introduction logique à une théorie déductive quelconque. Bibliothèque du Congrès International de Philosophie, Volume III, Logique et histoire des sciences, Librairie Armand Colin, Paris1901, pp. 309–365.” In: *Journal of Symbolic Logic* 40.3 (1975), pp. 473–527. DOI: [10.1017/S0022481200053603](https://doi.org/10.1017/S0022481200053603).
- [20] *Bubble Tea framework*. <https://github.com/charmbracelet/bubbletea>.
- [21] Massimo Lauria. *cnfgen*. <https://massimolauria.net/cnfgen/>.
- [22] Matthew W. Moskewicz et al. “Chaff: engineering an efficient SAT solver”. In: *Proceedings of the 38th Annual Design Automation Conference*. DAC ’01. Las Vegas, Nevada, USA: Association for Computing Machinery, 2001, pp. 530–535. ISBN: 1581132972. DOI: [10.1145/378239.379017](https://doi.org/10.1145/378239.379017). URL: <https://doi.org/10.1145/378239.379017>.